OFFICIAL · SENSITIVE

GY

Website Security Assessment Report

Ministry of Local Government & Regional Development — an external, non-intrusive security review of the ministry web portal conducted ahead of the Government of Guyana assuming hosting responsibility.

ASSESSED TARGET	kishanac4.sg-host.com
ASSESSMENT DATE	10 June 2026
ASSESSMENT TYPE	Black-box · non-intrusive
PLATFORM UNDER REVIEW	WordPress · Elementor · SiteGround
OVERALL POSTURE	Needs hardening before launch
FINDINGS	2 High · 4 Medium · 1 Low
PREPARED FOR	MLGRD / Government of Guyana
CLASSIFICATION	OFFICIAL · SENSITIVE

DOCUMENT CONTROL

Classification & document control

This page records the formal control information for this report — its classification, version, ownership and authorised distribution. The report is to be handled in line with its **OFFICIAL · SENSITIVE** classification: it names a live administrator account, a personal email address and specific technical weaknesses, and must therefore be shared only with personnel who have a legitimate need to act on it.

Document control

FIELD	VALUE
Document title	MLGRD Website Security Assessment Report
Classification	OFFICIAL · SENSITIVE
Version	1.0 (final)
Issue date	10 June 2026
Author / role	Security Assessment — prepared for MLGRD
Assessed system	WordPress staging instance of the MLGRD website — `https://kishanac4.sg-host.com/`
Assessment window	Single-day passive review, 10 June 2026
Distribution	MLGRD senior management; designated ICT / hosting team; appointed technical partner. Not for public release.
Review status	Final — issued for action

Version history

VERSION	DATE	AUTHOR	SUMMARY OF CHANGE
0.1	10 Jun 2026	Security Assessment	Initial draft of findings and evidence capture.
1.0	10 Jun 2026	Security Assessment	Final report issued to MLGRD: full findings, methodology, remediation roadmap, CVE watch-list and rebuild recommendation.

Handling instructions

- Store and transmit only through Government-approved channels; do not forward to personal accounts.
- The administrator login name and personal email quoted in this report are **live** at the time of writing — treat them as sensitive until Finding F-01 is remediated.
- Redact Appendix A (raw evidence) before any wider or external circulation.

CONTENTS

Table of contents

1	Executive Summary	4
2	Introduction	6
3	Scope & Authorisation	7
4	Methodology	8
5	Detailed Findings (F-01 – F-07)	9
5.1	F-01 — Username & email enumeration / PII disclosure	9
5.2	F-02 — Missing HTTP security headers	10
5.3	F-03 — Software / version disclosure	11
5.4	F-04 — Public, indexable staging environment	12
5.5	F-05 — Exposed authentication surface, no MFA / rate-limiting	13
5.6	F-06 — WordPress MCP / AI adapter endpoint exposed	14
5.7	F-07 — Lower-severity hygiene issues	15
6	Security Control Assessment	16
7	Recommendation: Static Rebuild	17
8	Remediation Roadmap	18
9	CVE Watch-list	19
10	Conclusion	20
11	Glossary	21
A	Appendix A — Evidence	22
B	Appendix B — References	23

How to read this report. Section 1 is written for senior decision-makers and can be read on its own. Sections 2–4 explain what was done and how. Section 5 is the technical core, with one clearly-labelled finding per issue (description, evidence, impact, likelihood, severity and remediation). Sections 6–9 turn the findings into a prioritised action plan and a patch watch-list. A plain-English glossary (Section 11) defines every technical term used.

SECTION 1

Executive summary

The Ministry of Local Government & Regional Development (MLGRD) website is a standard **WordPress** build — theme *Hello Elementor*, the page-builders *Elementor* and *ElementsKit*, plus a suite of Hostinger plugins — hosted on **SiteGround** behind an nginx web server. This assessment was carried out ahead of the Government of Guyana taking over hosting of the site, to establish its security posture before any production launch.

The hosting platform applies several sound baseline protections: sensitive files, directory listings, the legacy `xmlrpc.php` interface and the `TRACE` method are all blocked, and privileged administrative interfaces require authentication. **However, the application leaks information that materially lowers the cost of attacking it, and it ships none of the modern HTTP security headers that browsers rely on to protect visitors.** Two issues are rated **High** severity.

7	2	0/6	8
Finding groups identified	High-severity issues	Security headers present	Good controls confirmed

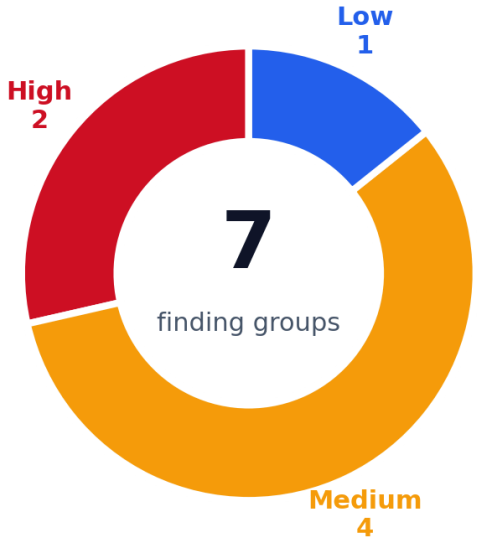

Figure 1 — Distribution of findings by severity.

Headline risk posture

The single most important issue is that **anyone, with no login, can retrieve the administrator's username and a second user's personal email address** from the site (F-01). Combined with an exposed login page that shows no two-factor authentication or rate-limiting (F-05), this gives an attacker a ready-made target for password-guessing and phishing. The absence of security headers (F-02) leaves every visitor's browser without standard protections.

Findings summary

ID	FINDING	SEVERITY
F-01	Username & email enumeration — admin login `kishanachang` and a personal email leaked unauthenticated	HIGH
F-02	No HTTP security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy and Permissions-Policy all absent)	HIGH
F-03	Software / version disclosure easing targeted CVE matching against a frequently-exploited plugin ecosystem	MEDIUM
F-04	Public, indexable staging environment with real ministry content on a non-government domain + leftover test page	MEDIUM
F-05	Login surface exposed with no visible two-factor authentication or rate-limiting, combined with known usernames	MEDIUM
F-06	WordPress MCP / AI adapter endpoint exposed (currently authentication-gated) — broad, destructive-capable surface	MEDIUM
F-07	Hygiene issues: open `wp-cron`, no `security.txt`, cookie missing `SameSite`, attribution leak	LOW

SECTION 1 (CONTINUED)

Top recommendations

The findings point to a single, twofold conclusion. **First**, if the existing WordPress site is to go live, it must be hardened beforehand (Section 8). **Second**, and more strongly, the Government should prefer the **static rebuild** already prepared for this project (Section 7), which structurally removes most of the attack surface described in this report.

#	PRIORITY RECOMMENDATION	ADDRESSES
1	Close all forms of user / email enumeration and rotate the administrator username away from `kishanachang`.	F-01
2	Deploy the full set of HTTP security headers, including HSTS, at the web-server or CDN edge.	F-02
3	Remove the staging site from search-engine indexes, place it behind authentication, and delete leftover test pages.	F-04
4	Enforce two-factor authentication and login rate-limiting on all administrative accounts.	F-05
5	Patch WordPress core, the theme and every plugin to current versions; remove unused plugins.	F-03
6	Disable the MCP / AI adapter and application passwords unless they are genuinely required in production.	F-06
7	**Strategically:** migrate to the static rebuild and serve production only from an official `*.gov.gy` domain with a valid certificate.	All

Bottom line for decision-makers. None of the issues found indicate that the site has been breached. They are, however, exactly the conditions that make a future breach *easy* — and the site carries the name and content of a Government ministry, which raises both the likelihood of being targeted and the reputational cost of an incident. The remediation work is well understood and achievable in days, not months. The static rebuild eliminates whole categories of this risk permanently and is the recommended path to a secure launch.

What was assessed

A single public WordPress staging instance was reviewed using read-only techniques only — HTTP fingerprinting, inspection of publicly reachable endpoints, a review of security headers and TLS, and mapping of disclosed software versions against published vulnerabilities. No exploitation, password attack, fuzzing, automated scanning or state-changing request was performed. The full scope and rules of engagement are set out in Section 3.

SECTION 2

Introduction

2.1 Background to the engagement

The Government of Guyana is preparing to take over the hosting of the website for the Ministry of Local Government & Regional Development. The site currently runs as a WordPress instance on a third-party commercial host (SiteGround) under a temporary staging address, `kishanac4.sg-host.com`. Before any production cutover — and certainly before the site is served from an official Government domain — it is prudent to understand its security posture: what an outside observer can learn about it, what protections it does and does not provide, and what would need to change to host it safely.

This report documents an independent, external security assessment commissioned for that purpose. It was deliberately conducted in a **non-intrusive** manner, using only information that any member of the public could obtain, so that the live site was never placed at risk during the review.

2.2 Why this matters

A Government ministry website is not an ordinary site. It carries the authority and identity of the State; citizens reasonably trust that what it publishes is genuine and that interacting with it is safe. That trust makes such sites attractive targets:

- **Impersonation & misinformation.** An attacker who can alter content — or who can clone a publicly indexable copy — can publish false notices, fraudulent forms, or phishing pages under the ministry's name.
- **Credential compromise.** If an administrator account is taken over, the attacker inherits full control of the site's content and configuration.
- **Pivot to citizens.** Personal data disclosed by the site (such as the staff email found in this review) can be used to target individuals with convincing, tailored phishing.
- **Reputational and political cost.** A defacement or data incident on a Government site attracts disproportionate public and media attention.

Because the Government will own the consequences of any incident once it hosts the site, the objective of this assessment is straightforward: to make sure the site is brought to an appropriate standard of security *before* it goes live under a Government domain, and to recommend the architecture that keeps it that way with the least ongoing effort.

2.3 Target fingerprint

For reference, the table below summarises what the assessment was able to determine about the technology stack from publicly available information alone.

PROPERTY	VALUE (AS DISCLOSED PUBLICLY)
Web server	nginx (SiteGround managed hosting)
Platform	WordPress (meta generator reports `WordPress 7.0` — treat as reported)
Theme	Hello Elementor v3.4.9 (disclosed via readable `style.css`)
Page builder	Elementor 4.1.1 (+ feature flags in meta generator); ElementsKit
Other plugins	Hostinger suite: easy-onboarding, AI assistant, amplitude (analytics), reach (CRM/email), tools; Elementor Pro namespaces present
REST namespaces	`wp/v2`, `elementor/v1`, `elementor-pro/v1`, `elementskit/v1`, `mcp`, `hostinger-*/v1`, `wp-abilities/v1`
Hosting artifacts	`X-Httpd-Modphp: 1`, `Host-Header` hash, `X-Proxy-Cache-Info` (SiteGround)
Developer attribution	User `kishanachang` profile URL `http://kishanachang.wpendgame.com` (a WordPress migration/staging service)

SECTION 3

Scope & authorisation

3.1 Assessed target

The assessment was limited to a single system: the public WordPress staging instance of the MLGRD website at `https://kishanac4.sg-host.com/`, together with the publicly reachable endpoints served under that hostname (for example its `wp-json` REST interface, sitemaps and login pages).

3.2 In scope

- HTTP/HTTPS response behaviour and security headers of the target hostname.
- Publicly reachable, unauthenticated endpoints (REST API collections, sitemaps, author archives, login and password-reset pages, `robots.txt`, `wp-cron.php`).
- TLS configuration and HTTP-to-HTTPS redirection behaviour.
- Software and version information voluntarily disclosed by the application.
- Mapping of the disclosed technology stack against published vulnerabilities (CVEs).

3.3 Out of scope

- Any authenticated or administrative functionality (no credentials were held or used).
- The underlying SiteGround hosting infrastructure, network, or other tenants on the platform.
- Other domains, sub-domains or services not served under the target hostname.
- Social engineering of staff, physical security, and denial-of-service testing.
- Source-code review of the WordPress core, theme or plugins.

3.4 Rules of engagement

This review was strictly read-only and non-intrusive. The following constraints were applied throughout:

- Only `GET`, `HEAD`, `OPTIONS` and `TRACE` requests were issued; no payloads or form data were submitted.
- **No exploitation** of any vulnerability was attempted. Impacts in this report are assessed from configuration and disclosed information only.
- No password attacks, brute-forcing, credential-stuffing or login attempts were made.
- No fuzzing, automated vulnerability scanning, or high-volume requesting was performed.
- No state-changing request (anything that could create, modify or delete data) was sent.

As a result, the assessment carried negligible risk to the availability or integrity of the live site. The trade-off is that some issues can only be confirmed definitively from inside an authenticated session or with the host's cooperation — those are flagged in the relevant findings (for example, the exact installed plugin versions behind the CVE watch-list in Section 9).

3.5 Authorisation

This assessment was conducted as a pre-hosting security review for the Government of Guyana, using only techniques available to any member of the public against a publicly published website. No special access, credentials, or privileged position was used or required.

SECTION 4

Methodology

4.1 Approach

The assessment followed a **passive reconnaissance** approach: the security posture of the site was inferred from the information it volunteers and the way it responds to ordinary, well-formed requests, rather than by attacking it. This mirrors the first stage of how a real attacker profiles a target, but stops short of any action that could affect the system. The work proceeded in four stages:

1. **Fingerprinting.** Identify the web server, platform, theme, plugins and versions from response headers, the HTML `generator` tag, readable asset files and REST API namespaces.
2. **Endpoint inspection.** Request publicly reachable endpoints (REST collections, sitemaps, author archives, login and password-reset pages, `robots.txt`, common sensitive paths) and record their status codes and content.
3. **Header & transport review.** Examine the HTTP security headers, cookie attributes, TLS usage and HTTP-to-HTTPS redirection.
4. **CVE mapping.** Match the disclosed software stack against published vulnerabilities using public databases, to produce a patch-priority watch-list.

4.2 Tools & techniques

Standard, widely-available techniques were used — command-line HTTP clients to retrieve headers and endpoint responses, a browser to confirm rendering and redirects, and public vulnerability databases (**WPScan**, **Patchstack** and the CVE catalogue) for the version mapping. No intrusive scanners or exploitation frameworks were employed.

4.3 How severity was rated

Each finding is rated **High**, **Medium** or **Low** using a qualitative model that weighs two factors:

- **Likelihood / ease of exploitation** — how easy it is for an attacker to abuse the weakness, and how widely the technique is known and automated.
- **Business impact** — the potential consequences for the Ministry if the weakness were abused, including content integrity, confidentiality of data, and reputation.

Plotting each finding on these two axes gives the risk matrix below. Items in the upper-right quadrant — high likelihood and high impact — warrant the most urgent attention. This scoring is intended for prioritisation and is qualitative; it is not a formal CVSS score, although indicative CVSS figures for the relevant plugin CVEs are provided in Section 9.

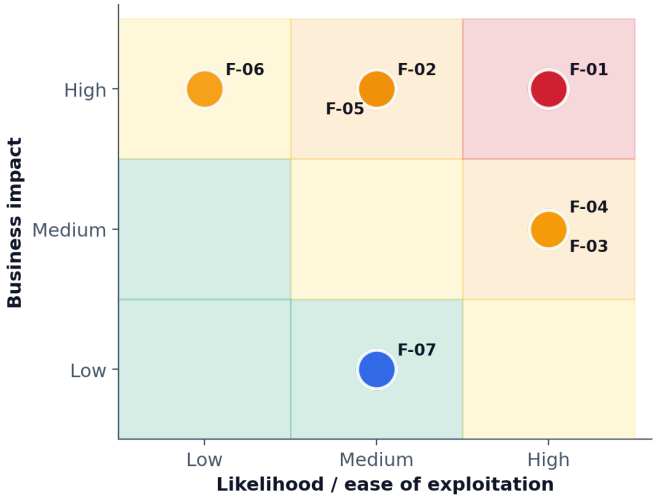

Figure 2 — Risk matrix: each finding plotted by likelihood (horizontal) against business impact (vertical).

Why F-01 sits top-right. Anyone can retrieve the administrator's login name and a second user's personal email from a single unauthenticated request. Combined with the exposed login page (F-05), that turns a two-unknown problem (username *and* password) into a one-unknown problem — the classic set-up for credential-stuffing and targeted phishing.

Background shading on the matrix indicates the combined risk level of each cell: green = low, amber = elevated, red = high.

SECTION 5 · DETAILED FINDINGS

5.1 F-01 — Username & email enumeration / PII disclosure

What is user enumeration? Many attacks need two things: a valid username and its password. "User enumeration" is any technique that lets an outsider obtain valid usernames without logging in. Once an attacker has a confirmed username, they only have to guess the password — and they can also use it to craft convincing, personalised phishing emails. Leaking usernames therefore removes half of the protection that login credentials are supposed to provide.

F-01 · Username & email enumeration / PII disclosure

HIGH

DESCRIPTION — The site exposes its list of registered authors to anyone, with no authentication, through several independent channels. The exposed data includes the login name of an **administrator** account and a user slug from which a **personal email address** can be reconstructed.

EVIDENCE —

- `GET /wp-json/wp/v2/users` returns `200` with the full author list: `id:1` → name and slug `kishanachang` (an administrator login), and `id:3` → slug `darrenxmhindsgmail-com`, which reconstructs the email **darrenxmhinds@gmail.com**.
- `GET /?author=1` → `301` redirect to `/author/kishanachang/`, confirming the username.
- `GET /wp-sitemap-users-1.xml` lists `/author/kishanachang/`.

BUSINESS IMPACT — Valid usernames are the first half of every credential attack. One account is a personal Gmail (a phishing target and a credential-stuffing pivot); the other is the site administrator. This converts a "guess username and password" problem into a "guess password only" problem and enables highly targeted spear-phishing against named individuals.

LIKELIHOOD	BUSINESS IMPACT	SEVERITY
High	High	HIGH

REMEDIATION

1. Disable REST user enumeration for unauthenticated requests — either via a security plugin (Wordfence, Solid Security) or by filtering `rest_endpoints` to remove `/wp/v2/users` for guests.
2. Block `?author=N` author-scan redirects and remove the users sitemap (the `wp_sitemaps_add_provider` filter).
3. Set display names so they differ from login names; never derive a username or email from a public slug.
4. Rotate the administrator username away from `kishanachang`, and advise the owner of the leaked personal email to be alert to targeted phishing.

SECTION 5 · DETAILED FINDINGS

5.2 F-02 — Missing HTTP security headers

What are HTTP security headers? When a browser loads a page, the web server can attach a set of instructions — "headers" — that tell the browser how to protect the visitor. They prevent the page being framed by a malicious site (clickjacking), stop the browser guessing file types (MIME-sniffing), force the use of a secure encrypted connection (HSTS), and limit the damage a content-injection bug could do (CSP). These headers cost nothing to add and are considered a basic standard for any modern site, especially a Government one.

F-02 · Missing HTTP security headers

HIGH

DESCRIPTION — The site's responses include **none** of the modern HTTP security headers. Visitors' browsers are therefore left without the standard protections these headers provide.

EVIDENCE — Responses for `/` contain none of: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` or `Permissions-Policy`. HTTP does redirect (`301`) to HTTPS, but with no HSTS there is an SSL-strip window on a visitor's first visit.

BUSINESS IMPACT — No clickjacking protection (no `X-Frame-Options` / `frame-ancestors`); MIME-sniffing is possible (`X-Content-Type-Options` absent); no Content-Security-Policy to contain any cross-site scripting (XSS) — particularly relevant given the Elementor stack's history of stored-XSS bugs; referrer leakage to third parties; and no transport pinning, leaving a window for connection downgrade on first contact.

LIKELIHOOD	BUSINESS IMPACT	SEVERITY
Medium	High	HIGH

REMEDIATION — Add the following headers at the edge (nginx, SiteGround or Cloudflare) for every response:

1. `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`
2. A tested `Content-Security-Policy` appropriate to the site's assets.
3. `X-Frame-Options: SAMEORIGIN` and `X-Content-Type-Options: nosniff`.
4. `Referrer-Policy: strict-origin-when-cross-origin`.
5. A least-privilege `Permissions-Policy` disabling features the site does not use.

SECTION 5 · DETAILED FINDINGS

5.3 F-03 — Software / version disclosure

Why version disclosure matters. Software flaws are catalogued publicly as "CVEs", each tied to specific affected versions. If a site openly advertises exactly which versions of WordPress and its plugins it runs, an attacker can look up the matching known flaws in seconds — without touching the site — and go straight to a working attack. Hiding versions does not fix the flaws, but it removes an easy shortcut and is a sensible layer of defence.

F-03 · Software / version disclosure

MEDIUM

DESCRIPTION — The application openly discloses the versions of its core components, allowing precise matching against published vulnerabilities.

EVIDENCE — The HTML meta generator leaks `WordPress 7.0` and `Elementor 4.1.1` (plus feature flags); the theme's `style.css` is readable and reveals `Hello Elementor 3.4.9`; and per-asset `?ver=` query strings expose component versions.

BUSINESS IMPACT — An attacker can match the exact stack to public CVEs without interacting with the application. The Elementor ecosystem is heavily targeted: recent examples include **CVE-2025-49387** (unauthenticated file-upload to remote code execution in an Elementor Forms upload add-on), **CVE-2025-8081** (Elementor arbitrary file read), **CVE-2025-13067** (Royal Addons RCE), **CVE-2026-4659** (Unlimited Elements local file inclusion) and several King-Addons criticals scoring up to CVSS 10.0. The full watch-list is in Section 9.

LIKELIHOOD	BUSINESS IMPACT	SEVERITY
High	Medium	MEDIUM

REMEDIATION

1. Strip the `generator` meta tag and remove asset `?ver=` query strings.
2. Suppress publicly readable theme/plugin version files where possible.
3. Treat version-hiding as defence-in-depth only — it is **not** a substitute for keeping every component fully patched (see Section 9).

SECTION 5 · DETAILED FINDINGS

5.4 F-04 — Public, indexable staging environment

What is a staging environment? A "staging" site is a work-in-progress copy used to build and test a website before it goes live. It should normally be hidden from the public and from search engines. Here, the staging copy of the ministry site is fully public, crawlable, and hosted on a commercial domain rather than a Government one.

F-04 · Public, indexable staging environment

MEDIUM

DESCRIPTION — A live, crawlable staging site is serving real ministry content on a non-government domain, and contains leftover scaffold pages from development.

EVIDENCE — The site is fully crawlable (`robots.txt` allows general crawling and a sitemap is published) and serves real ministry content on a non-government domain (`*.sg-host.com`). A leftover `/test-about/` page is published.

BUSINESS IMPACT — Search engines will index an unofficial copy of Government content, causing citizen confusion and duplicate content, and providing a ready-made template for phishing and impersonation. Staging sites also commonly run looser configuration and debug features, and leftover test pages signal an unfinished, unreviewed surface.

LIKELIHOOD	BUSINESS IMPACT	SEVERITY
High	Medium	MEDIUM

REMEDIATION

1. Until launch, apply `noindex` plus HTTP Basic authentication or an IP allow-list to the staging host so it is not publicly reachable or indexable.
2. Remove `test-about` and any other scaffold or test pages.
3. Serve production only from the official `*.gov.gy` domain with a valid certificate, and retire the `sg-host.com` address once cutover is complete.

SECTION 5 · DETAILED FINDINGS

5.5 F-05 — Exposed authentication surface, no MFA / rate-limiting

Brute-force and credential-stuffing. If a login page is openly reachable, has no limit on how many attempts can be made, and offers no second factor, an attacker can try thousands of password guesses (brute-force) or replay passwords leaked from other breaches (credential-stuffing). Two-factor authentication (2FA) and rate-limiting are the standard defences.

F-05 · Exposed login surface, no visible MFA / rate-limiting

MEDIUM

DESCRIPTION — The WordPress login and password-reset pages are openly reachable, with no observable second factor or attempt-limiting — and the usernames are already known from F-01.

EVIDENCE — `wp-login.php` → `200`; `?action=lostpassword` → `200`; registration (`?action=register`) → `302` (appears disabled — good). No challenge, throttling or 2FA was observed.

BUSINESS IMPACT — The site is a direct target for credential-stuffing and brute-force attacks, made materially easier by the known usernames from F-01. The lost-password flow can also assist user enumeration.

LIKELIHOOD	BUSINESS IMPACT	SEVERITY
Medium	High	MEDIUM

REMEDIATION

1. Enforce two-factor authentication (2FA) for all administrator and editor accounts.
2. Add login rate-limiting and lockout (Wordfence, Solid Security or a Limit-Login plugin).
3. Restrict `wp-admin` and `wp-login.php` by IP allow-list, or place them behind SSO/VPN.
4. Require strong, unique passwords, and consider a custom (non-default) login path.

SECTION 5 · DETAILED FINDINGS

5.6 F-06 — WordPress MCP / AI adapter endpoint exposed

What is an MCP / AI adapter? "MCP" (Model Context Protocol) is a newer mechanism that lets AI tools and automation drive a system through a single bridge endpoint. On a website, such a bridge can be very powerful — potentially able to change or delete content. Because these components are new and not yet well-tested, exposing one even in a locked-down state widens the attack surface.

F-06 · WordPress MCP / AI adapter endpoint exposed

MEDIUM

DESCRIPTION — An AI/automation control endpoint is publicly discoverable. It is currently authentication-gated (good), but its presence — including a destructive verb — broadens the attack surface, and application passwords are enabled on the site.

EVIDENCE — `/wp-json/mcp` is publicly listed and exposes `/wp-json/mcp/mcp-adapter-default-server` accepting `POST`, `GET` and `DELETE`. Unauthenticated calls currently return `401 rest_forbidden` (good), but the AI/agent control surface is discoverable and includes a destructive verb.

BUSINESS IMPACT — An AI "tools" bridge is a powerful, fast-evolving attack surface. If any credential or **application password** were to leak, an attacker could potentially drive AI/automation tools — including content deletion or configuration change — through a single endpoint. New MCP / abilities plugins are immature and under-audited.

LIKELIHOOD	BUSINESS IMPACT	SEVERITY
Low	High	MEDIUM

REMEDIATION

1. If MCP / AI features are not required in production, **disable** the MCP adapter, the `hostinger-ai-assistant` and the `wp-abilities` plugins.
2. If they are required, restrict the route to authenticated administrators over an allow-listed network.
3. Audit-log every call to the adapter.
4. Disable application passwords unless they are explicitly needed.

SECTION 5 · DETAILED FINDINGS

5.7 F-07 — Lower-severity hygiene issues

The following issues are individually low-risk, but each is a small piece of housekeeping that a Government site should attend to. Collectively they reflect general configuration hygiene and are inexpensive to fix.

F-07 · Lower-severity hygiene

LOW

FINDINGS & FIXES

- `wp-cron.php` → **200.** Anyone can trigger the site's scheduled tasks, a mild resource-abuse / denial-of-service amplification vector. *Fix:* set `define('DISABLE_WP_CRON', true)`, run a real system cron, and block external access to `wp-cron.php`.
- **Public REST collections.** `wp/v2/media`, `comments`, `pages` and `types` all return `200` (standard behaviour, but `media` enables full attachment/author enumeration). *Fix:* restrict where feasible and ensure no draft or PII content is exposed.
- **No** `/.well-known/security.txt` (`404`). *Fix:* publish a `security.txt` with a disclosure contact — this is expected of a Government site.
- **Login cookie lacks** `SameSite`**.** `Secure` and `HttpOnly` are set, but `SameSite` is absent. *Fix:* add `SameSite=Lax` (or `Strict`).
- **Developer-attribution leak.** The profile URL `kishanachang.wpendgame.com` reveals the build tooling and provenance. *Fix:* sanitise user profile URLs.

LIKELIHOOD	BUSINESS IMPACT	SEVERITY
Medium	Low	LOW

Note. Although these are individually minor, the `security.txt` and cookie `SameSite` items in particular are quick wins that bring the site in line with baseline expectations for a public-sector website.

SECTION 6

Security control assessment

Not everything about this site is weak — and it is important to be balanced. The hosting platform provides a number of solid baseline controls that are confirmed to be working. The chart below contrasts the controls that are present and effective (green) against those that are missing or weak (red). The host does well on file and endpoint lockdown and on transport security, but HTTP security headers and authentication hardening are largely absent.

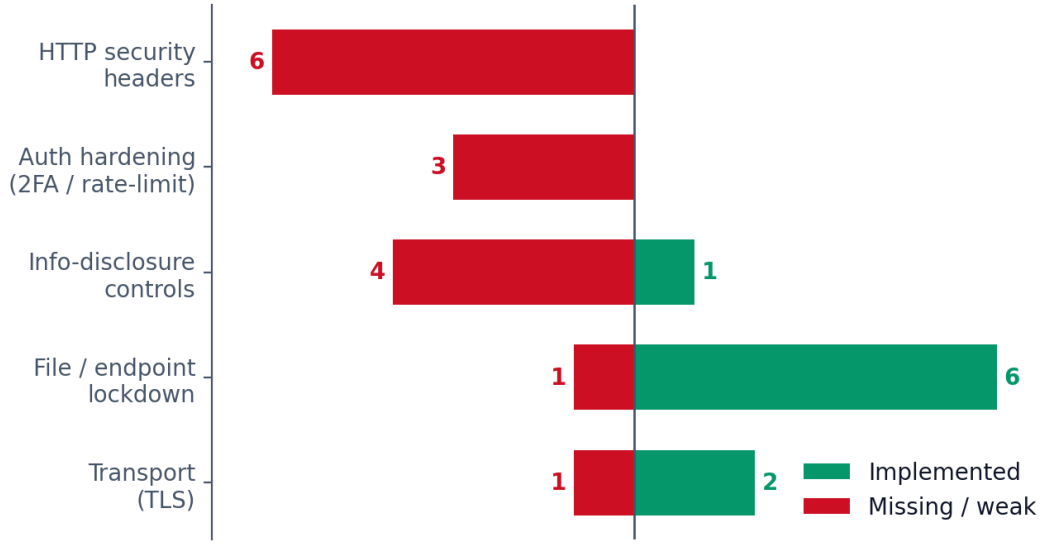

Figure 3 — Security control coverage: implemented controls (right) versus missing or weak controls (left).

Confirmed good (8 controls).

- HTTPS enforced (`301` → HTTPS); valid TLS in use.
- `xmlrpc.php` blocked (`403`) — blocks pingback DDoS and XML-RPC brute-force.
- Sensitive files all `403`: `.git/config`, `.env`, `wp-config` backups, `backup.zip`, `debug.log`, `.user.ini`.
- Directory listing disabled for uploads, plugins and themes.
- Privileged REST routes require auth (`settings`, `plugins`, `themes` → `401`).
- Application-passwords page requires auth; MCP adapter calls → `401`.
- `TRACE` → `405` (no cross-site tracing); CORS is not wildcard-open.
- User registration appears disabled (`302`).

Missing / weak (12+ controls).

- All six modern security headers absent (F-02).
- No two-factor authentication and no login rate-limiting (F-05).
- REST user enumeration open; software versions disclosed (F-01 / F-03).
- `wp-cron` externally triggerable; no `security.txt` (F-07).
- Login cookie missing `SameSite` (F-07).
- Public, indexable staging surface (F-04); MCP adapter discoverable (F-06).

The pattern is consistent: **infrastructure-level controls supplied by the host are good, but application-level and edge configuration is where the gaps lie.** That is encouraging, because the gaps are precisely the things that can be configured quickly — and that the static rebuild (Section 7) provides by default.

SECTION 7

Recommendation: the static rebuild

The strongest single recommendation in this report is architectural. The Government should host a **statically-exported site** (the replacement already prepared for this project) in place of the WordPress instance. A static site has no server-side code, no database, no administrative panel and no plugins on the host — it is simply a set of pre-built HTML, CSS and JavaScript files served from a content delivery network. Structurally, it **cannot** have most of the vulnerabilities in this report, because the components that produce them do not exist.

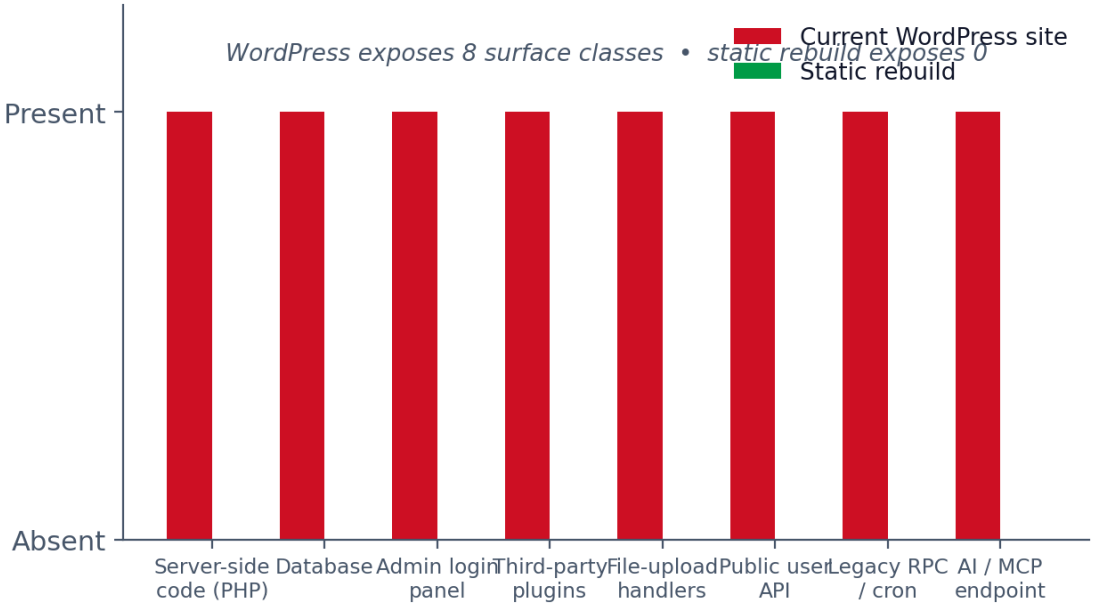

Figure 4 — Attack-surface comparison: the WordPress site exposes eight classes of surface; the static rebuild exposes none.

EXISTING WORDPRESS RISK	STATIC REBUILD
PHP + database + admin panel + 10+ plugins running on the host	No server code, no database, no admin, no plugins — only static HTML/CSS/JS
User / email enumeration (F-01)	No user accounts on the host — nothing to enumerate
Plugin RCE / file-upload / LFI CVEs (F-03)	No plugins and no upload handlers — the entire CVE class is removed
`xmlrpc`, `wp-cron`, MCP adapter, login surface (F-05 / F-06 / F-07)	None of these endpoints exist
Missing security headers (F-02)	Ships HSTS + CSP + X-Frame-Options + nosniff + Referrer-Policy + Permissions-Policy by default

Additional hardening built into the rebuild

- Subresource-integrity-friendly bundling of assets.
- No third-party trackers by default (consent-gated if any are added).
- Form submissions handled via a vetted endpoint, with no PHP mailer to exploit.
- Pinned dependencies with automated auditing (`npm audit` / Dependabot).
- A published `security.txt` and deployment on an official domain with a valid certificate.

Net effect. The attack surface drops from "a full content-management system" to "a CDN serving files." This does not remove the need for the immediate fixes in Section 8 (which protect the site during any interim period), but it makes the secure state the default rather than something that must be continuously maintained.

SECTION 8

Remediation roadmap

The actions below are prioritised into three phases — **Immediate** (before any production cutover), **Short-term**, and **Ongoing**. Owner suggestions and effort are indicative.

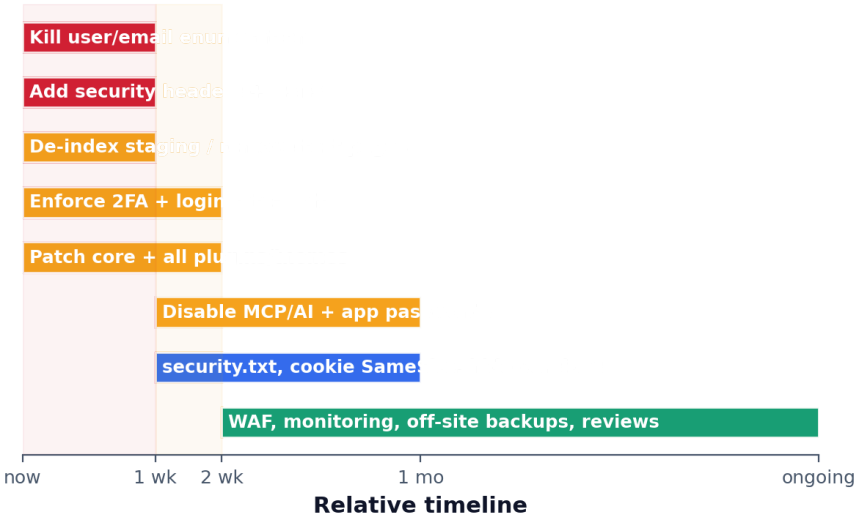

Figure 5 — Indicative remediation timeline by phase.

ACTION	FINDING	SUGGESTED OWNER	EFFORT
Immediate — before any production cutover			
Kill user / email enumeration; rotate the `kishanachang` admin name	F-01	ICT / WordPress admin	Low (hours)
Add the full security-header set + HSTS at the edge	F-02	Hosting partner / CDN	Low (hours)
De-index staging (noindex + auth/IP allow-list); delete `test-about`	F-04	ICT / WordPress admin	Low (hours)
Enforce 2FA + login rate-limiting; restrict `wp-admin`	F-05	ICT / WordPress admin	Low–Medium (1 day)
Patch WordPress core + all plugins/themes; remove unused plugins	F-03	ICT / WordPress admin	Medium (1–2 days)
Short term — within ~2 weeks			
Disable MCP / AI / abilities + application passwords unless needed; audit-log if kept	F-06	ICT / WordPress admin	Low (hours)
Disable HTTP `wp-cron`; publish `security.txt`; set cookie `SameSite`; strip version strings	F-03 / F-07	ICT / hosting partner	Low (hours)
Ongoing — continuous			
Managed WAF (SiteGround / Cloudflare), file-integrity monitoring, off-site backups, least-privilege accounts, quarterly review	All	ICT + hosting partner	Ongoing
Migrate production to the static rebuild on an official `*.gov.gy` domain	All	Programme / ICT	Project

SECTION 9

CVE watch-list

The exact installed plugin builds could not be confirmed without authentication, so the list below is a **patch-priority watch-list** rather than a confirmation that the site is vulnerable to each item. The administrator should check the installed versions in `wp-admin → Plugins` against WPScan and Patchstack, and patch or remove anything affected. The chart shows the indicative maximum CVSS severity for each affected component family.

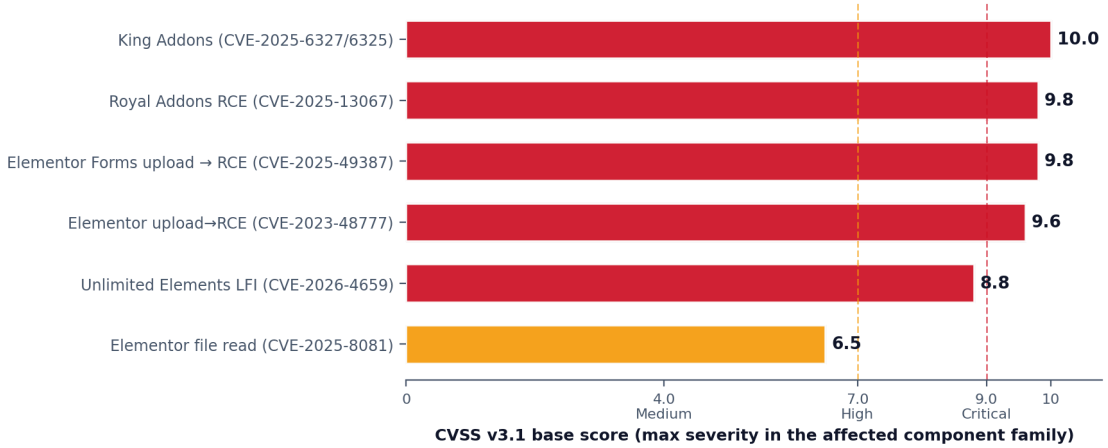

Figure 6 — Indicative CVSS v3.1 base scores for the most serious CVEs in the affected component families.

CVE	COMPONENT	IMPACT	ACTION
CVE-2025-49387	Elementor Forms upload add-on	Unauthenticated file-upload → remote code execution (RCE)	Patch / remove add-on
CVE-2025-8081	Elementor (≤ 3.30.2)	Arbitrary file read	Update Elementor
CVE-2023-48777	Elementor (historic)	Authenticated file-upload → RCE	Confirm patched
(stored XSS)	Elementor Pro (< 3.29.1)	Stored cross-site scripting (XSS)	Update Elementor Pro
(class)	ElementsKit	Recurring authenticated file-upload / LFI / SSRF issues across releases	Update / remove if unused
CVE-2025-6327 / 6325	King Addons	Critical (CVSS up to 10.0)	Remove if installed/unused
CVE-2025-13067	Royal Addons	Remote code execution (RCE)	Remove if installed/unused
CVE-2026-4659	Unlimited Elements	Local file inclusion (LFI)	Remove if installed/unused

Action. Update WordPress core and every plugin/theme to the current release, enable auto-updates for security releases, and remove any unused plugins — each Hostinger or Elementor add-on left installed is additional attack surface. Version-hiding (F-03) is not a substitute for patching.

SECTION 10

Conclusion

The MLGRD website, in its current WordPress form, is **not ready to be hosted under a Government domain without remediation**. Nothing in this assessment indicates that the site has been compromised; rather, the issues found are the conditions that make a future compromise easy — and the site's Government identity raises both the chance of being targeted and the cost of an incident.

The two High-severity findings — open user/email enumeration (F-01) and the complete absence of HTTP security headers (F-02) — should be closed before any cutover, alongside de-indexing the staging site, enforcing two-factor authentication, and patching the plugin stack. Encouragingly, the hosting platform already provides a solid set of baseline protections, and the application-level gaps that remain can be configured quickly.

The clearest path to a secure launch is the **static rebuild**. By removing the server-side code, database, administrative panel and plugins entirely, it eliminates whole classes of risk permanently rather than requiring them to be continuously managed. Combined with deployment on an official `*.gov.gy` domain, a valid certificate, security headers shipped by default, and pinned, audited dependencies, it reduces the attack surface from "a full content-management system" to "a CDN serving files."

Recommended path to a secure launch.

1. Apply the Immediate remediation actions (Section 8) to the current site to protect it during any interim period.
2. Stand up the static rebuild on an official Government domain with a valid certificate and default security headers.
3. Cut over to the rebuild, retire the `sg-host.com` staging address, and decommission the WordPress instance.
4. Maintain the Ongoing controls — monitoring, backups, least-privilege accounts and periodic review.

With these steps, the Ministry can launch its website on a footing appropriate to a Government service — one that protects both the integrity of its content and the citizens who rely on it.

SECTION 11

Glossary

Plain-English definitions of the technical terms used in this report.

HSTS	HTTP Strict Transport Security. A header that tells browsers to only ever connect to the site over an encrypted (HTTPS) connection, closing the window for a "downgrade" attack on first visit.
CSP	Content-Security-Policy. A header that restricts what content a page is allowed to load and run, which helps contain the damage from content-injection bugs such as XSS.
XSS	Cross-Site Scripting. A flaw that lets an attacker inject malicious scripts into a web page, which then run in other visitors' browsers — for example to steal sessions or deface content.
CSRF	Cross-Site Request Forgery. A trick that causes a logged-in user's browser to perform an unwanted action on a site without their intent. The `SameSite` cookie attribute helps prevent it.
RCE	Remote Code Execution. The most serious class of flaw: it lets an attacker run their own commands on the server, effectively taking it over.
LFI	Local File Inclusion. A flaw that lets an attacker make the application read or include files on the server that it should not expose.
SSRF	Server-Side Request Forgery. A flaw that tricks the server into making requests to systems the attacker could not otherwise reach (such as internal services).
REST API	A standard web interface (here, WordPress's `wp-json`) that lets software read and write data over HTTP. Useful, but it can over-share information if not locked down.
CVE	Common Vulnerabilities and Exposures. A public catalogue that gives each known software flaw a unique identifier (e.g. CVE-2025-49387).
CVSS	Common Vulnerability Scoring System. A 0–10 scale that rates how severe a CVE is; 9.0+ is "Critical".
WAF	Web Application Firewall. A protective layer in front of a site that filters out malicious requests before they reach the application.
MFA / 2FA	Multi-Factor / Two-Factor Authentication. Requiring a second proof of identity (such as a code from a phone) in addition to a password, so a stolen password alone is not enough to log in.
User enumeration	Any technique that reveals valid usernames to an outsider, making password-guessing and phishing much easier.
Clickjacking	An attack that hides a target site inside an invisible frame on a malicious page, tricking a user into clicking something they did not intend. The `X-Frame-Options` header prevents it.
MIME-sniffing	When a browser guesses a file's type instead of trusting the server's label, which can cause it to run a file as a script. The `X-Content-Type-Options: nosniff` header prevents it.
Application password	A special, separate password WordPress can issue so that software/automation can access the site's API. Convenient, but a leak can grant broad access.
MCP	Model Context Protocol. A newer mechanism that lets AI tools and automation control a system through a single bridge endpoint — powerful, and therefore sensitive.
Brute-force	Repeatedly guessing passwords (or other secrets) until one works. Rate-limiting and 2FA are the standard defences.
Credential-stuffing	Re-using username/password pairs leaked from other breaches against a site, in the hope that people reused them.
Staging	A work-in-progress copy of a website used to build and test it before it goes live. It should normally be hidden from the public.

APPENDIX A

Evidence (selected raw responses)

The following raw evidence was captured during the assessment using unauthenticated `GET` / `HEAD` / `OPTIONS` / `TRACE` requests only. No payloads were submitted. Sensitive values are quoted here because they are material to Finding F-01; this appendix should be redacted before any wider circulation.

A.1 — Root response headers (no security headers present)

```
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 10 Jun 2026 16:25:25 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Vary: Accept-Encoding
Link: <https://kishanac4.sg-host.com/wp-json/>; rel="https://api.w.org/"
Link: <https://kishanac4.sg-host.com/wp-json/wp/v2/pages/14>; rel="alternate"; type="application/json"
Link: <https://kishanac4.sg-host.com/>; rel=shortlink
X-Httpd-Modphp: 1
Host-Header: 8441280b0c35cbc1147f8ba998a563a7
X-Proxy-Cache-Info: DT:1

# NOTE: none of Strict-Transport-Security, Content-Security-Policy, X-Frame-Options,
#       X-Content-Type-Options, Referrer-Policy or Permissions-Policy are present (F-02).
```

A.2 — `GET /wp-json/wp/v2/users` (unauthenticated user enumeration, F-01)

```
[
  {
    "id": 3,
    "name": "Darren Hinds",
    "link": "https://kishanac4.sg-host.com/author/darrenxmhindsgmail-com/",
    "slug": "darrenxmhindsgmail-com"   # reconstructs darrenxmhinds@gmail.com
  },
  {
    "id": 1,
    "name": "kishanachang",
    "url":  "http://kishanachang.wpendgame.com",   # developer-attribution leak (F-07)
    "link": "https://kishanac4.sg-host.com/author/kishanachang/",
    "slug": "kishanachang"   # administrator login name
  }
]
```

A.3 — Author-scan confirmation

```
GET /?author=1           ->  301  Location: /author/kishanachang/
GET /wp-sitemap-users-1.xml  ->  200  lists /author/kishanachang/
```

All probes were unauthenticated and read-only. Full header dumps and endpoint responses are retained under `_audit/` (e.g. `root_headers.txt`, `users_rest.json`).

APPENDIX B

References

Public sources used for the technology mapping, CVE watch-list and remediation guidance.

Vulnerability databases

- WPScan — WordPress vulnerability database. https://wpscan.com/ (e.g. plugin record https://wpscan.com/plugin/elementor-pro/).
- Patchstack — WordPress & plugin vulnerability intelligence. https://patchstack.com/database/
- CVE / NVD — the public CVE catalogue and National Vulnerability Database. https://nvd.nist.gov/

CVEs referenced in this report

- CVE-2025-49387 — Elementor Forms upload add-on, unauthenticated upload → RCE.
- CVE-2025-8081 — Elementor (≤ 3.30.2), arbitrary file read.
- CVE-2023-48777 — Elementor, historic authenticated upload → RCE.
- CVE-2025-6327 / CVE-2025-6325 — King Addons, critical (CVSS up to 10.0).
- CVE-2025-13067 — Royal Addons, remote code execution.
- CVE-2026-4659 — Unlimited Elements, local file inclusion.

Standards & guidance

- OWASP Secure Headers Project — guidance on HTTP security headers (HSTS, CSP, X-Frame-Options, etc.).
- OWASP Top 10 — common web application security risks.
- RFC 9116 — the `security.txt` standard for security disclosure contacts.
- Mozilla MDN Web Docs — reference for HTTP headers and cookie `SameSite` attributes.

Note on sources. CVE identifiers and severities in this report are indicative and should be verified against the vulnerability databases above using the *installed* plugin and theme versions, which could not be confirmed during this non-intrusive assessment.

End of report · MLGRD Website Security Assessment · v1.0 · 10 June 2026 · OFFICIAL · SENSITIVE